

Første celle downloader jeg datasættet

```
21  
12 sek. !pip install -q kagglehub tensorflow scikit-learn matplotliblib  
  
import kagglehub  
  
path = kagglehub.dataset_download("vip000001/new-plant-diseases-dataset")  
  
print(f"Datasæt downloadet til: {path}")  
  
import os  
for item in os.listdir(path):  
    print(f"- {item}")
```

... Using Colab cache for faster access to the 'new-plant-diseases-dataset' dataset.
Dataset downloadet til: /kaggle/input/new-plant-diseases-dataset
- New Plant Diseases Dataset(Augmented)
- new plant diseases dataset(augmented)
- test

Lister alle 38 klasser og antal billeder per klasse for at tjekke datasættets størrelse og balance.

```
import os  
  
#Sætter base path til en variable samt train mappen og validation mappen  
base_path = "/kaggle/input/new-plant-diseases-dataset/New Plant Diseases Dataset(Augmented)/New Plant Diseases Dataset(Augmented)"  
train_dir = os.path.join(base_path, "train")  
valid_dir = os.path.join(base_path, "valid")  
  
#Sætter variablen classes til en alfabetisk sortet liste af de klasser der optræder i train mappen  
classes = sorted(os.listdir(train_dir))  
print(f"Antal klasser: {len(classes)}\n")  
  
#Printer alle klasserne så man kan se dem for sig og hvor mange billeder de indeholder  
print("Alle klasser:")  
for c in classes:  
    n_images = len(os.listdir(os.path.join(train_dir, c)))  
    print(f" {c:50s} ({n_images} billeder)")
```

... Antal klasser: 38

Alle klasser:

Apple__Apple_scab	(2016 billeder)
Apple__Black_rot	(1987 billeder)
Apple__Cedar_apple_rust	(1760 billeder)
Apple__healthy	(2008 billeder)
Blueberry__healthy	(1816 billeder)
Cherry_(including_sour)__Powdery_mildew	(1683 billeder)
Cherry_(including_sour)__healthy	(1826 billeder)
Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot	(1642 billeder)
Corn_(maize)__Common_rust	(1907 billeder)
Corn_(maize)__Northern_Leaf_Blight	(1908 billeder)
Corn_(maize)__healthy	(1859 billeder)
Grape__Black_rot	(1888 billeder)
Grape__Esca_(Black_Measles)	(1920 billeder)
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	(1722 billeder)
Grape__healthy	(1692 billeder)
Orange__Haunglongbing_(Citrus_greening)	(2010 billeder)
Peach__Bacterial_spot	(1838 billeder)
Peach__healthy	(1728 billeder)
Pepper,_bell__Bacterial_spot	(1913 billeder)
Pepper,_bell__healthy	(1988 billeder)
Potato__Early_blight	(1939 billeder)
Potato__Late_blight	(1939 billeder)
Potato__healthy	(1824 billeder)
Raspberry__healthy	(1781 billeder)
Soybean__healthy	(2022 billeder)
Squash__Powdery_mildew	(1736 billeder)
Strawberry__Leaf_scorch	(1774 billeder)
Strawberry__healthy	(1824 billeder)

Indlæser billeder fra mapperne, opdeler i træning og validation, og definerer image augmentation (flip, rotation, zoom) for at reducere overfitting.

```
# Importerer TensorFlow og layers-modulet fra Keras
import tensorflow as tf
from tensorflow.keras import layers

# Billedstørrelse skal være 224x224 fordi MobileNetV2 er trænet på den størrelse
IMG_SIZE = (224, 224)
# Batch size = antal billeder modellen behandler ad gangen per vægtopdatering
BATCH_SIZE = 32

# Indlæser træningsbilleder fra mappen - klasse-labels udledes automatisk fra mappenavne
# Billederne resizes til 224x224, grupperes i batches af 32, og blandes tilfældigt
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=True, seed=42 # seed=42 gør shuffle reproducerbar
)

# Indlæser validation-billeder - bruges KUN til evaluering, ikke til træning
# Vi blander ikke (shuffle=False) da rækkefølgen er ligegyldig ved evaluering
val_ds = tf.keras.utils.image_dataset_from_directory(
    valid_dir,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=False
)

# Gemmer listen af klassenavne og antal klasser - skal bruges senere i modellen
class_names = train_ds.class_names
num_classes = len(class_names)

# Augmentation pipeline - udvider datasettet kunstigt og reducerer overfitting
# Anvendes KUN på træningsdata, ikke validation
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"), # Spejlvender tilfældigt - meningsfuldt for blade
    layers.RandomRotation(0.15), # Roterer +/- 54 grader (15% af 360)
    layers.RandomZoom(0.1), # Zoomer +/- 10%
])

# Prefetch lader CPU forberede næste batch mens GPU træner på den nuværende - fjerner ventetid
# Vi cacher 1000 da decoded dataset fylder over 40 GB - for stort til Colabs RAM
train_ds = train_ds.prefetch(tf.data.AUTOTUNE)
val_ds = val_ds.prefetch(tf.data.AUTOTUNE)

print(f"Klasser: {num_classes}")
```

Found 70295 files belonging to 38 classes.
Found 17572 files belonging to 38 classes.
Klasser: 38

Bygger en CNN ved at bruge en MobileNetV2 fortrænet på ImageNet som base, fryser de fortrænede lag og træner et nyt klassifikations-hoved oven på til de 38 plantesygdomme.

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras import layers, models

# Base-model: MobileNetV2 fortrænet på ImageNet (transfer learning)
base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False, # fjern det sidste lag - vi bygger vores eget
    weights="imagenet"
)
base_model.trainable = False # fryser vægtene - vi træner kun vores nye lag

# Byg model
inputs = layers.Input(shape=(224, 224, 3))
x = data_augmentation(inputs) # augmentation (kun aktiv under træning)
x = preprocess_input(x) # MobileNetV2 forventer pixels i [-1, 1]
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x) # reducerer overfitting
outputs = layers.Dense(num_classes, activation="softmax")(x)

model = models.Model(inputs, outputs)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# Træn modellen
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5, # 5 epochs er nok med transfer learning
    verbose=1
)
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 2s 0us/step

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
sequential (Sequential)	(None, 224, 224, 3)	0
true_divide (TrueDivide)	(None, 224, 224, 3)	0
subtract (Subtract)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 38)	48,678

Total params: 2,306,662 (8.80 MB)

Trainable params: 48,678 (190.15 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Epoch 1/5
2197/2197 — 405s 181ms/step — accuracy: 0.8603 — loss: 0.4866 — val_accuracy: 0.9222 — val_loss: 0.2548
Epoch 2/5
2197/2197 — 159s 72ms/step — accuracy: 0.9205 — loss: 0.2448 — val_accuracy: 0.9346 — val_loss: 0.2041
Epoch 3/5
2197/2197 — 158s 72ms/step — accuracy: 0.9284 — loss: 0.2162 — val_accuracy: 0.9356 — val_loss: 0.2019
Epoch 4/5
2197/2197 — 158s 72ms/step — accuracy: 0.9312 — loss: 0.2052 — val_accuracy: 0.9318 — val_loss: 0.2073
Epoch 5/5
2197/2197 — 158s 72ms/step — accuracy: 0.9352 — loss: 0.1952 — val_accuracy: 0.9444 — val_loss: 0.1669

Måler validation accuracy og loss, og tjekker om modellen opfylder succeskriteriet.

```
#Evaluering på validation-sættet
print("\nEvaluerer model...")
val_loss, val_accuracy = model.evaluate(val_ds, verbose=1)

print("\n" + "="*50)
print("RESULTATER - Plant Disease Classification")
print("="*50)
print(f"Validation Accuracy: {val_accuracy:.4f} (krav: > 0.70)")
print(f"Validation Loss: {val_loss:.4f}")
print(f"Antal klasser: {num_classes}")
print(f"Træningsbilleder: 70295")
print(f"Validationsbilleder: 17572")
print("="*50)

# Tjek om kravene er opfyldt
acc_ok = val_accuracy > 0.70
if acc_ok:
    print("MODEL OPFYLDER KRAV!")
else:
    print("Accuracy for lav = model opfylder ikke krav!")
```

```
Evaluering på validation-sættet
550/550 31s 56ms/step - accuracy: 0.9444 - loss: 0.1669

=====
RESULTATER - Plant Disease Classification
=====
Validation Accuracy: 0.9444 (krav: > 0.70)
Validation Loss: 0.1669
Antal klasser: 38
Træningsbilleder: 70295
Validationsbilleder: 17572
=====
MODEL OPFYLDER KRAV!
```